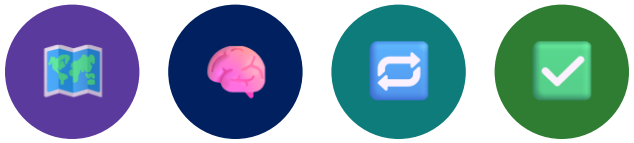
*"Automate. Provision. Configure. Build infrastructure faster, smarter, at scale."*

MODULE 45

# Infrastructure as Code with AI Assistance

Provision infrastructure with Terraform, configure it with Ansible, and use AI assistance to accelerate, validate, and improve every decision -- with humans always accountable.



TEKsystems Global Services

# WORKFORCE DEVELOPMENT

# MODULE 45 OVERVIEW

*Provision infrastructure with Terraform, configure it with Ansible, and use AI assistance responsibly -- with humans accountable for every apply.*

The Northstar CRM team wants faster, repeatable environment setup for dev/test/stage -- but AI-generated infrastructure can be syntactically plausible while insecure, destructive, expensive, or non-idempotent.

## DURATION & LAB

### 1 Hour Theory

Infrastructure as Code, Terraform provisioning, Ansible configuration management, and AI-assisted review discipline.

### 2 Hours Lab

lab45-crm: AI-drafted Terraform + Ansible sketches for Northstar CRM, validated, corrected, and documented.

### Scenario

Northstar CRM non-production environments (crm-dev / crm-test) -- never real customer data in IaC.

## MODULE ARC

### Understand IaC

Why automation matters, what IaC is, and the end-to-end workflow.

### Master Terraform & Ansible

Provisioning, state, configuration management, and idempotence.

### Use AI Responsibly

Draft with AI, review critically, and document every decision in the lab.

**KEY TAKEAWAY** Total time: 1 hour theory + 2 hours hands-on lab = 3 hours. Syntactically valid, AI-generated Terraform that opens a public database still fails the lab -- humans stay accountable.

# WHY INFRASTRUCTURE AUTOMATION MATTERS

*Modern applications demand infrastructure that is consistent, scalable, secure, and available on demand -- manual processes can't keep up.*

| The Challenge with Manual Infrastructure | The Solution: Infrastructure Automation |
|---|---|
| **Time-consuming and error-prone.** | Automate provisioning and configuration. |
| **Inconsistent environments.** | Consistent, repeatable environments. |
| **Difficult to scale quickly.** | Scale infrastructure in minutes. |
| **Hard to track changes and audit.** | Version-controlled and auditable changes. |
| **Higher risk of security misconfigurations.** | Built-in security and compliance. |
| **Increased operational costs.** | Lower costs, higher efficiency. |

## BUSINESS IMPACT

### Faster Time to Market

Teams focus on features, not managing servers.

### Scalability on Demand

Provision and scale to meet business needs.

### Reliability & Security

Consistent configs reduce error and risk.

### Cost Optimization

Right-size infrastructure to reduce waste.

**KEY TAKEAWAY** With AI assistance, you can write better IaC code, detect misconfigurations, and automate with confidence. Automate today, deliver value every day.

# MODULE LEARNING OBJECTIVES

*By the end of this module, you will be able to provision, configure, and manage infrastructure using Terraform, Ansible, and AI assistance.*

- **1. Understand IaC —** explain the concept, benefits, and workflow of Infrastructure as Code and why automation matters.
- **2. Work with Terraform Fundamentals —** understand Terraform architecture, providers, resources, and configuration files.
- **3. Execute the Terraform Workflow —** use terraform init, plan, apply, and destroy to provision and manage infrastructure.
- **4. Manage Terraform State —** understand state files, remote state concepts, and best practices for state management.
- **5. Understand Ansible Fundamentals —** explain Ansible architecture, inventory, playbooks, and how Ansible works.
- **6. Automate Configuration Management —** create Ansible playbooks to automate configuration tasks with idempotent operations.
- **7. Combine Terraform and Ansible —** integrate Terraform for provisioning and Ansible for configuration into one workflow.
- **8. Apply Best Practices —** implement industry best practices for secure, scalable, repeatable infrastructure automation.

**KEY TAKEAWAY** You will gain practical skills to automate infrastructure, improve reliability, reduce manual effort, and accelerate delivery with AI-powered assistance.

# WHAT IS INFRASTRUCTURE AS CODE?

*IaC is the practice of managing and provisioning infrastructure through machine-readable code instead of manual processes.*

## HOW IaC WORKS


Define infra in code -> the tool reads it -> provisions or updates infra -> deployed, consistent, and repeatable.

## WHY IT MATTERS

- **Consistency —** same code = same environment every time.
- **Automation —** eliminates manual steps and human errors.
- **Scalability —** provision and scale resources quickly and reliably.
- **Version Controlled —** track changes, review, and roll back using Git.
- **Cost Efficiency —** optimize resource usage and reduce operational costs.

## MANUAL VS INFRASTRUCTURE AS CODE

| Manual Infrastructure | Infrastructure as Code |
|---|---|
| **Manual, repetitive tasks** | Automated and repeatable |
| **Prone to human error** | Reduced errors |
| **Inconsistent environments** | Consistent environments |
| **Hard to track changes** | Version controlled |
| **Slow to scale** | Fast and scalable |

## EXAMPLES OF IaC TOOLS

**Terraform** **Ansible** **AWS CloudFormation** **Pulumi** **Azure Bicep**

Key idea: you describe infrastructure in code (declarative or imperative); the tool reads it and automatically creates, updates, or destroys the infrastructure for you.

**KEY TAKEAWAY** IaC brings software engineering best practices -- code, version control, automation, testing, deployment -- to infrastructure. Write once, use everywhere.

# BENEFITS OF INFRASTRUCTURE AUTOMATION

*Infrastructure automation enables teams to deliver consistent, secure, and scalable environments faster while reducing costs and manual effort.*

### 1. Speed & Agility

Provision and configure infrastructure in minutes, not hours or days.

### 2. Consistency

Use code to ensure the same environment every time across dev/test/prod.

### 3. Reliability

Reduce human error and misconfigurations with repeatable, versioned infra.

### 4. Scalability

Easily scale resources up or down without manual intervention.

### 5. Security & Compliance

Enforce policies as code; detect drift and remediate automatically.

### 6. Cost Efficiency

Optimize resource usage, eliminate waste, only pay for what you need.

### 7. Visibility & Auditability

Track every change and audit infrastructure like application code.

### 8. Better Collaboration

Enables collaboration between operations, developers, and security teams.

**KEY TAKEAWAY** Automate once, use everywhere, deliver value always. Less manual work, more innovation, better outcomes -- forming the backbone of modern DevOps and cloud-native environments.

# IaC WORKFLOW OVERVIEW

*Infrastructure as Code follows a consistent workflow from writing code to managing and maintaining your infrastructure.*


A continuous feedback loop: after Monitor & Maintain, the cycle returns to Write Code -- not a one-time event.

## KEY PRINCIPLES

- **Declarative —** define the desired state, not the steps.
- **Idempotent —** apply changes repeatedly with the same result.
- **Versioned —** track all changes like application code.
- **Reproducible —** recreate environments consistently.
- **Automated —** reduce manual work and human errors.

## BUSINESS VALUE

- Faster delivery.
- Consistent & reliable environments.
- Lower costs.
- Better visibility and stronger collaboration.
- Audit & compliance built in.

Outcome: infrastructure that is consistent, scalable, secure, and delivered efficiently with code.

**KEY TAKEAWAY** IaC brings DevOps best practices to infrastructure: Code, Automate, Validate, Deploy, Operate, Improve -- a continuous feedback loop, not a one-time event.

# AI ASSISTANCE IN INFRASTRUCTURE AS CODE

*AI accelerates drafting Terraform and Ansible -- but humans remain accountable for exposure, cost, state security, and idempotence.*

## WHERE AI ASSISTANCE HELPS

- **Code Generation —** scaffold IaC templates, modules, and boilerplate.
- **Code Explanation —** explain existing IaC code and resources.
- **Refactoring & Optimization —** improve structure, reduce cost and complexity.
- **Policy & Compliance —** generate and validate policies, checks, and guardrails.
- **Troubleshooting —** analyze errors, suggest fixes and best practices.
- **Documentation —** auto-generate documentation and diagrams.

## THE HUMAN ACCOUNTABILITY GATE

- Write bounded, contract-first prompts that state assumptions and forbid secrets/public exposure.
- Review generated Terraform/Ansible critically -- 'it linted' is not a review.
- Never apply because 'AI said so' -- a human reads every plan action.
- Reject or harden at least one unsafe AI suggestion, with rationale.
- Document every AI decision in a dated AI review record.

### Example: a bounded, contract-first prompt

```
Generate Terraform for a non-prod Kubernetes namespace crm-${environment}
with labels application=crm. No public LoadBalancer/DB. No plaintext secrets.
Pin provider versions. List every assumption made.
Include a human review checklist alongside the generated code.
```

**KEY TAKEAWAY** Syntactically valid Terraform that opens a public database still fails review. AI drafts scaffolding; humans stay accountable for every apply.

# TRY IT YOURSELF — EXERCISE 1: INFRA CONTRACT

*Reinforces: writing a bounded infra contract before any AI prompt -- what's allowed, what's forbidden, before Terraform begins.*

## STEPS (notes/lab45-infra-contract.md)

| Step | What To Do |
|---|---|
| **1 — Contract Fields** | State env names (crm-dev/crm-test), region, network, runtime, DB posture, tags, cost limits. |
| **2 — Forbidden List** | Write the reference table: allowed (sketches, .tfvars.example) vs forbidden (real keys, tfstate, PII). |
| **3 — Tags** | Propose tags: application=crm, environment=dev, owner=<your note>. |
| **4 — Data Rule** | State explicitly: fixtures CUS-1001 / CUS-1002 stay in app labs -- never in Terraform/Ansible. |

## Reference: allowed vs forbidden in IaC

```
Allowed:   network/runtime sketches, tfvars.example, inventory.example.yml, tags
Forbidden: real cloud keys, terraform.tfstate, customer PII, unreviewed public DB
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-infra-contract.md (workspace: examples/module-45-exercises).

# WHAT IS TERRAFORM?

*Terraform is an open-source Infrastructure as Code tool by HashiCorp that lets you define, provision, and manage infrastructure using declarative configuration files.*

## KEY IDEA

- You describe the desired state of your infrastructure in code, and Terraform figures out how to create or update it.

### Example resource block

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
  }
}
```

## WHY USE TERRAFORM?

**Multi-Cloud**

Works with AWS, Azure, Google Cloud, and more.

**Consistent**

Repeatable deployments every time.

**Version Controlled**

Store infra as code in Git for collaboration.

**Cost Efficient**

Automate and optimize to reduce manual effort.

## WORKS WITH 100+ PROVIDERS

AWS | Azure | Google Cloud | Kubernetes | VMware | ...and many more

**KEY TAKEAWAY** Terraform helps teams build, change, and version infrastructure safely and efficiently using code -- enabling automation, consistency, and reliability at scale.

# TERRAFORM ARCHITECTURE

*Terraform uses a modular architecture to define, provision, and manage infrastructure across multiple cloud providers.*

1. **Terraform Code** — Written by developer.
2. **Terraform Core** — Parses, plans, manages.
3. **Provider Plugins** — AWS, Azure, GCP...
4. **Cloud APIs** — Provider calls.
5. **Infra Resources** — VMs, networks, storage.

## TERRAFORM CORE RESPONSIBILITIES

- **Parses HCL —** reads and validates .tf files written in HashiCorp Configuration Language.
- **Creates Execution Plan —** shows what will be created, changed, or destroyed before applying.
- **Builds Dependency Graph —** determines resource relationships and correct order of operations.
- **Manages Resource Lifecycle —** creates, updates, and deletes resources to reach the desired state.
- **Tracks Infrastructure State —** stores and refreshes state to keep infrastructure in sync.

## COMPONENTS

| Component | Role |
|---|---|
| **Terraform CLI** | Command-line interface for running Terraform commands. |
| **Terraform Core** | Dependency graph, planning, resource management. |
| **Terraform State** | Stores current state; maps real resources; enables operations. |
| **Provider Plugins** | Translate HCL into cloud-specific API calls. |

**KEY TAKEAWAY** Terraform separates infrastructure logic from cloud-specific implementations using provider plugins, enabling consistent, multi-cloud automation.

# PROVIDERS AND RESOURCES

*Terraform uses providers to interact with APIs of different platforms. Providers expose resources that represent infrastructure objects.*

## KEY CONCEPTS

- **Provider = Plugin —** communicates with a platform (cloud, SaaS, or on-prem).
- **Resource = Infra Object —** the individual objects Terraform manages (VM, VPC, bucket, DB).
- **Multiple Providers —** use several providers in one configuration for multi-cloud.
- **Lifecycle Management —** Terraform handles create, read, update, delete, and keeps state in sync.

## HOW TERRAFORM WORKS (HIGH-LEVEL FLOW)


Providers translate Terraform configuration into platform-specific API calls, while resources represent the infrastructure objects being created and managed.

## EXAMPLE RESOURCES BY PROVIDER

| Provider | Resources (examples) |
|---|---|
| **AWS** | EC2, VPC, S3, RDS, IAM |
| **Azure** | Virtual Machines, Storage Account, Virtual Network |
| **Google Cloud** | Compute Engine, Cloud Storage, GKE |
| **Kubernetes** | Pods, Deployments, Services, ConfigMaps |

## GITHUB PROVIDER (BEYOND CLOUD)

Providers aren't only clouds: 1000+ providers exist for databases, monitoring, networking, DevOps tools, and SaaS platforms like GitHub (repositories, teams, actions).

**KEY TAKEAWAY** Providers translate Terraform configuration into platform-specific API calls, while resources are the infrastructure objects being created and managed.

# TERRAFORM CONFIGURATION FILES

*Terraform uses Human-Readable Configuration Language (HCL) in .tf files to define infrastructure in a declarative, version-controlled way.*

## ANATOMY OF A .tf FILE

- **Provider Block —** specifies the cloud or platform to use.
- **Resource Block —** defines infrastructure resources to create or manage.
- **Variable —** makes values reusable and configurable.
- **Output —** exposes information about provisioned resources.

**example.tf**

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "data" {
  bucket = var.bucket_name
  tags = {
    environment = var.environment
  }
}

output "bucket_name" {
  value = aws_s3_bucket.data.bucket
}
```

## COMMON FILE STRUCTURE

| File | Purpose |
|---|---|
| **main.tf** | Main resources |
| **providers.tf** | Provider config |
| **variables.tf** | Input variables |
| **outputs.tf** | Output values |
| **.tfvars** | Variable values |

## BEST PRACTICES

- Use meaningful resource names and tags.
- Use variables for values that change per environment.
- Organize code using modules and folders.
- Never hardcode secrets in .tf files.
- Always use Git for version control and reviews.

**KEY TAKEAWAY** Terraform configuration files are the source of truth for your infrastructure. Write once, use everywhere -- HCL is declarative and version-controlled.

# TERRAFORM INIT

*The terraform init command initializes a Terraform working directory -- always the first command to run, and safe to run multiple times.*

### 1. Initializes

Creates the .terraform directory to store plugins and modules.

### 2. Downloads Plugins

Downloads required provider plugins defined in the configuration.

### 3. Installs Modules

Downloads and installs modules from the Terraform Registry or other sources.

### 4. Configures Backend

Prepares the backend for storing the state file (local or remote).

**$ terraform init**

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.30.0...

Terraform has been successfully initialized!
```

## WHEN TO USE

- When starting a new Terraform project.
- After cloning a repository.
- When provider or module requirements change.
- When backend configuration changes.

**KEY TAKEAWAY** terraform init is always the first command to run and sets up everything needed to work with Terraform. It is safe to run multiple times.

# TERRAFORM PLAN

*The terraform plan command creates an execution plan -- a read-only preview of what Terraform will do before making any changes.*

### 1. Reads Config & State

Loads .tf files and the current state from the backend.

### 2. Compares

Compares the desired state (config) with the current state.

### 3. Creates Execution Plan

Determines the actions needed to reach the desired state.

### 4. Safe & Non-Destructive

No changes are made during the plan operation.

**$ terraform plan**

```
# aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami           = "ami-0c55b159cbfafe1f0"
      + instance_type = "t3.micro"
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

## PLAN EXIT CODES

| # | Meaning |
|---|---|
| **0** | No changes -- infrastructure is up-to-date. |
| **1** | Changes present -- plan shows actions to be taken. |
| **2** | Error -- plan failed. |

**KEY TAKEAWAY** terraform plan is a read-only operation. It shows what will be added, changed, or destroyed -- always run plan before apply, and always read every action.

# TERRAFORM APPLY

*The terraform apply command executes the actions proposed in the plan and makes the real changes to your infrastructure.*

### 1. Creates Plan

Generates the plan of actions needed to reach the desired state.

### 2. Gets Approval

Asks for confirmation ("yes") to proceed with the changes.

### 3. Executes Actions

Performs create/update/destroy actions in the correct order.

### 4. Updates State

Updates the state file so infrastructure matches configuration.

**$ terraform apply**

```
Do you want to perform these actions?
  Only 'yes' will be accepted to approve.
  Enter a value: yes

aws_instance.web: Creating... [id=i-0abc123def4567890]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

## APPLY OPTIONS

| Command | Behavior |
|---|---|
| **apply** | Interactive -- asks for confirmation. |
| **apply -auto-approve** | Skips confirmation (CI/CD only). |
| **apply plan.out** | Applies a saved plan file. |

**KEY TAKEAWAY** terraform apply makes REAL changes and requires explicit approval unless auto-approved. Always review the plan before you apply -- plan, then approve, then apply.

# TERRAFORM DESTROY

*The terraform destroy command safely removes all resources managed by Terraform. Use with caution -- this action is irreversible.*

### 1. Creates Destroy Plan

Generates a plan of resources that will be destroyed.

### 2. Gets Confirmation

Asks for explicit confirmation before proceeding.

### 3. Destroys Resources

Removes resources in the correct order to handle dependencies.

### 4. Updates State

Removes destroyed resources from the state file.

**$ terraform destroy**

```
Terraform will perform the following actions:
  # aws_instance.web will be destroyed
  - resource "aws_instance" "web" { ... }

Do you really want to destroy all resources? Enter: yes
Destroy complete! Resources: 3 destroyed.
```

## SAFETY BEST PRACTICES

- Always run terraform plan before destroy.
- Use -target carefully -- can cause drift.
- Protect production with approval workflows.
- Keep state in a remote backend with locking.

**KEY TAKEAWAY** Caution: this action is irreversible. Destroyed resources cannot be recovered by Terraform -- review the plan, confirm intentionally, destroy only what you truly want removed.

# TERRAFORM STATE FILES

*Terraform state is a record of the infrastructure Terraform manages. It maps your real-world resources to your configuration.*

## WHAT IS A STATE FILE?

- A JSON file that stores the current state of your infrastructure.
- Tracks every resource Terraform creates and manages.
- Essential for planning, applying, and destroying accurately.

## STATE FILE CHARACTERISTICS

- Contains sensitive information (IDs, secrets, ARNs).
- Must be secured and access-controlled.
- Stores resource metadata and dependencies.
- Enables drift detection and recovery.
- Should be versioned and backed up.

## WHY STATE FILES MATTER

- **Source of Truth —** Terraform relies on state to know what exists in the real world.
- **Resource Mapping —** maps configuration to actual resources.
- **Change Detection —** enables Terraform to detect drift.
- **Safe Operations —** prevents accidental overwrites and inconsistencies.

## STORAGE (BACKENDS)

**Amazon S3** **Azure Storage**

**GCS** **Terraform Cloud**

## COMMON ISSUES

- **State Corruption —** may cause inconsistencies.
- **Manual Changes Outside Terraform —** leads to drift.
- **State Lost or Deleted —** can cause resource recreation risk.
- **Concurrent Changes Without Locking —** causes conflicts.

## BEST PRACTICES

- Use remote state for teams.
- Enable state locking.
- Encrypt state at rest.
- Limit access with IAM/RBAC.
- Backup regularly; never manually edit state.

**KEY TAKEAWAY** Treat your Terraform state like a database -- secure it, back it up, and manage it carefully. State is the source of truth for planning, applying, and destroying.

# REMOTE STATE CONCEPTS

*Remote state stores your Terraform state file in a remote backend instead of locally -- enabling collaboration, locking, security, and scalability.*

## WHY USE REMOTE STATE?

- **Collaboration —** teams can work together on the same infrastructure.
- **State Locking —** prevents concurrent operations and state corruption.
- **Security —** state is stored securely with encryption and access control.
- **Scalability & Reliability —** remote backends are durable, available, and versioned.

## POPULAR REMOTE BACKENDS

| Backend | Notes |
|---|---|
| **Amazon S3** | Durable, versioned; locking via DynamoDB. |
| **Azure Storage** | Blob versioning; locking via blob leases. |
| **Google Cloud Storage** | Strong consistency and versioning. |
| **Terraform Cloud** | Built-in state, locking, versioning, access mgmt. |

## STATE FLOW WITH REMOTE BACKEND


## Backend configuration example

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "envs/dev/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "terraform-lock"
    encrypt = true
  }
}
```

**KEY TAKEAWAY** Remote state = Collaboration + Safety (Locking) + Security + Reliability. The state file acts as the single source of truth for all plan/apply/destroy operations.

# STATE MANAGEMENT BEST PRACTICES

*Terraform state is critical. Following best practices ensures your infrastructure stays safe, consistent, secure, and reliable across teams.*

### 1. Use Remote State

Store state in a remote backend (S3, Azure Blob, GCS, Terraform Cloud) -- never locally in team environments.

### 2. Enable State Locking

Always enable locking (DynamoDB, blob leases) to prevent concurrent writes.

### 3. Enable Versioning

Turn on versioning to keep history and enable point-in-time recovery.

### 4. Encrypt State

Encrypt at rest and in transit (SSE-KMS, TLS).

### 5. Restrict Access

Apply least-privilege IAM policies -- only those who need it.

### 6. Do Not Commit State

Never commit .tfstate files to version control.

### 7. Backup Regularly

Automate backups and periodically test restore procedures.

### 8-10. Workspaces, No Manual Edits, Monitor

Isolate environments with workspaces; never hand-edit state; monitor access and changes.

**KEY TAKEAWAY** Golden Rule: treat your Terraform state like production data -- because it is. A corrupted or lost state can lead to outages or unwanted resource recreation.

# TRY IT YOURSELF — EXERCISE 2: TERRAFORM CHECKS

*Reinforces: fmt/init/validate/plan expectations and a remote-state narrative -- planning the checks before ever touching real cloud credentials.*

## STEPS (notes/lab45-terraform-checks.md)

| Step | What To Do |
|---|---|
| **1 — Command Order** | List: terraform fmt, init, validate, plan (or an instructor-safe substitute). |
| **2 — Read the Plan** | State explicitly: read create/destroy risk before any apply discussion. |
| **3 — State Narrative** | Write 3 bullets on encrypted remote state + locking, without backend credentials. |
| **4 — Evidence Path** | Note that sanitized plan snippets belong under notes/screenshots/lab-45/. |

## Command order to document

```
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
terraform plan -var='environment=dev' -out=tfplan
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-terraform-checks.md (workspace: examples/module-45-exercises).

# WHAT IS ANSIBLE?

*Ansible is an open-source IT automation tool that simplifies configuration management, application deployment, and task automation using agentless, human-readable YAML.*

## KEY FEATURES

- **Agentless Architecture —** no agents to install or manage on target systems.
- **Simple & Human-Readable —** uses YAML playbooks that are easy to learn and maintain.
- **SSH-Based Communication —** uses SSH (or WinRM for Windows) for secure communication.
- **Idempotent —** ensures systems remain in the desired state.

## SIMPLE PLAYBOOK EXAMPLE

```
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

Ansible pushes configuration and commands to managed nodes over SSH/WinRM and ensures the desired state is achieved.

## COMMON USE CASES

**Configuration Management** | **Infra Provisioning** | **App Deployment** | **Orchestration** | **Compliance & Security**

**KEY TAKEAWAY** Ansible makes automation simple, powerful, and scalable -- without the need for agents. Reduces manual effort, speeds provisioning, and ensures consistency.

# ANSIBLE ARCHITECTURE

*Ansible uses an agentless, control node-managed architecture to automate and manage infrastructure at scale.*

1. **Write Playbook** — YAML on control node.
2. **Read Inventory** — Know the managed nodes.
3. **Connect** — SSH / WinRM.
4. **Push & Execute** — Modules run tasks.
5. **Return Results** — Facts & status.

## CONTROL NODE VS MANAGED NODES

| Node | Holds / Is |
|---|---|
| **Control Node** | Ansible engine, playbooks, inventory, modules, plugins. |
| **Managed Nodes** | Linux/Unix, Windows, network devices, cloud instances. |

## COMMUNICATION

- **Linux/Unix —** SSH (port 22 by default), key-based authentication recommended.
- **Windows —** WinRM (port 5985 HTTP / 5986 HTTPS).

## ARCHITECTURE CHARACTERISTICS

- **Agentless —** no software to install on managed nodes.
- **Push-Based —** the control node pushes tasks to nodes.
- **Idempotent —** ensures desired state without side effects.
- **Cross-Platform —** Linux, Windows, network, cloud.

**KEY TAKEAWAY** Ansible's agentless architecture makes automation simple, secure, and easy to scale -- from a control node to hundreds or thousands of managed nodes.

# ANSIBLE INVENTORY FILES

*Ansible inventory defines the hosts and groups of hosts on which Ansible runs tasks -- the foundation for targeting systems.*

## STATIC INVENTORY — INI FORMAT

```
[webservers]
web1 ansible_host=10.0.1.11
web2 ansible_host=10.0.1.12

[dbservers]
db1  ansible_host=10.0.3.11

[all:children]
webservers
dbservers
```

## INVENTORY FILE ELEMENTS

- **Hosts —** individual machines or instances (e.g., web1, db1).
- **Groups —** a collection of hosts (e.g., webservers).
- **Variables —** key-value pairs defining connection or configuration settings.
- **Group Hierarchy —** groups can contain other groups ([parent:children]).
- **Special Groups —** all (all hosts), ungrouped (not in any group).

## USEFUL COMMANDS

- **Test connectivity —** ansible all -i inventory.ini -m ping
- **List hosts in a group —** ansible webservers -i inventory.ini --list-hosts

## BEST PRACTICES

- Group hosts logically.
- Use variables to avoid repetition.
- Use dynamic inventory for cloud environments.
- Keep sensitive data in Ansible Vault.
- Commit only inventory.example.yml -- never real host credentials.

**KEY TAKEAWAY** Inventory is the backbone of Ansible automation. Organize your hosts, use variables, and choose the right inventory type for scalable, maintainable automation.

# ANSIBLE PLAYBOOKS

*Playbooks are YAML files that define tasks to run on remote hosts. They describe WHAT you want to achieve -- Ansible handles HOW.*

## PLAYBOOK STRUCTURE

- **name —** name of the play.
- **hosts —** target hosts or groups.
- **become —** run with privilege escalation.
- **vars —** define variables.
- **tasks —** list of tasks to execute.
- **handlers —** tasks triggered by notifications.

## Example playbook (YAML)

```yaml
---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  vars:
    package_name: nginx
    service_name: nginx
  tasks:
    - name: Install Nginx package
      ansible.builtin.yum:
        name: "{{ package_name }}"
        state: present
    - name: Start and enable service
      ansible.builtin.service:
        name: "{{ service_name }}"
        state: started
        enabled: yes
  handlers:
    - name: Restart Nginx
      ansible.builtin.service:
        name: "{{ service_name }}"
        state: restarted
```

## COMMON MODULES

- **package —** install/remove packages.
- **copy —** copy files to remote hosts.
- **template —** copy files from Jinja2 templates.
- **service —** manage services.
- **user —** manage user accounts.

## USEFUL COMMANDS

- ansible-playbook site.yml
- ansible-playbook --syntax-check site.yml
- ansible-playbook --check site.yml (dry run)

## BENEFITS

- Human-readable, version-controllable.
- Reusable and modular.
- Idempotent and consistent.

**KEY TAKEAWAY** Ansible playbooks turn infrastructure tasks into automated, repeatable, and consistent operations. Running the same playbook again produces the same result.

# PROVISIONING VS CONFIGURATION MANAGEMENT

*Both are essential for automation, but they solve different problems at different stages of the infrastructure lifecycle.*

| Aspect | Infrastructure Provisioning | Configuration Management |
|---|---|---|
| **Purpose** | Creates and delivers infrastructure resources (VMs, networks, storage). | Configures and maintains software and settings on existing infrastructure. |
| **When It Runs** | Done once or infrequently (environment created/changed). | Runs continuously or frequently to keep the desired state. |
| **Common Tools** | Terraform, AWS CloudFormation, Azure ARM/Bicep, Pulumi. | Ansible, Puppet, Chef, SaltStack. |
| **Outcome** | New infrastructure (instances, VPCs, buckets, DBs). | Configured systems that follow the desired state (apps, configs, policies). |
| **Owned By** | Typically platform/DevOps/cloud teams. | Typically operations/SRE/DevOps teams. |

## HOW THEY WORK TOGETHER

1. **Provision Infra**
   Via Terraform.
2. **Infra Ready**
   Created and ready.
3. **Configure Systems**
   Via Ansible.
4. **Secure & Ready**
   Compliant, in service.

**KEY TAKEAWAY** Provisioning builds it. Configuration management keeps it running the right way, all the time. Together they deliver reliable, consistent, repeatable environments.

# AUTOMATING CONFIGURATION TASKS

*Automation helps you configure systems consistently, reduce manual effort, and ensure they remain in the desired state -- every time.*


## COMMON CONFIGURATION TASKS

- User & group management.
- Package management (install/update/remove).
- Service management (start/stop/enable).
- File & directory management, permissions.
- Security hardening (firewall, SSH, sudoers).
- Application configuration and templates.

## Example playbook snippet

```
- name: Configure Web Servers
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Ensure Nginx running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
      notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

## BENEFITS YOU GET

- Consistent configurations across all environments.
- Faster deployments and updates.
- Reduced manual errors and troubleshooting time.
- Easy rollback and version control with Git.
- Auditability and compliance enforcement.

**KEY TAKEAWAY** Ansible is idempotent -- running the same task multiple times produces the same result without changing the system if it is already in the desired state.

# IDEMPOTENT OPERATIONS

*Idempotency ensures that running the same operation multiple times produces the same result without unintended side effects.*

## IDEMPOTENCY IN ACTION — ENSURE NGINX IS INSTALLED

| 1st Run | 2nd Run | 3rd Run | Result |
|---|---|---|---|
| **changed (installed now)** | ok (already present) | ok (already present) | Same desired state every time -- safe to run repeatedly. |

## HOW ANSIBLE ACHIEVES IDEMPOTENCY

- **State Awareness —** modules check current state before making changes.
- **Declarative Modules —** describe desired state (state: present), not steps.
- **Smart Defaults —** modules return changed only when a change is made.
- **Accurate Reporting —** modules report facts about what changed (or not).

## IDEMPOTENT VS NON-IDEMPOTENT

### Good (module, declarative) vs bad (raw command)

```
# Idempotent -- checks state before acting
- name: Ensure Nginx is installed
  ansible.builtin.apt:
    name: nginx
    state: present

# Non-idempotent -- always re-runs, no state check
- name: Install Nginx (BAD)
  ansible.builtin.command: apt install -y nginx
```

## BEST PRACTICES FOR IDEMPOTENCY

- Use Ansible built-in modules instead of raw shell/command.
- Use state parameters (present, absent, started).
- Use templates, not copy, for config files.
- Test playbooks by running them multiple times.

**KEY TAKEAWAY** Idempotency is the foundation of reliable automation, self-healing systems, and DevOps at scale. Idempotent operations are safe to run again and again.

# TRY IT YOURSELF — EXERCISE 3: ANSIBLE IDEMPOTENCE

*Reinforces: naming idempotent modules/handlers for a CRM host sketch and proving a second run reports zero changes.*

## STEPS (notes/lab45-ansible-idempotence.md)

| Step | What To Do |
|---|---|
| **1 — Modules** | Name the modules/handlers you expect: package, service, copy/template, handler restart. |
| **2 — Second-Run Rule** | State: second run should be changed=0 when authorized; prove with lint/syntax first. |
| **3 — Ownership/Modes** | Note that file ownership and modes matter for application config files. |
| **4 — Inventory Hygiene** | Commit only inventory.example.yml -- never real host credentials. |

## Idempotent CRM host sketch (from the lab's worked example)

```
- name: Create CRM service account
  ansible.builtin.user: { name: crm, system: true, shell: /usr/sbin/nologin }
- name: Install runtime configuration
  ansible.builtin.template: { src: crm.env.j2, dest: /etc/crm/crm.env, mode: "0640" }
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-ansible-idempotence.md (workspace: examples/module-45-exercises).

# TERRAFORM AND ANSIBLE TOGETHER

*Terraform provisions infrastructure. Ansible configures it. Together they deliver end-to-end automation for consistent, reliable, repeatable environments.*

1. **Define Infra** — Terraform code.
2. **Provision** — terraform apply.
3. **Output Inventory** — IPs / hostnames.
4. **Configure** — Ansible connects.
5. **Desired State** — Ready for use.

## DYNAMIC INVENTORY WITH TERRAFORM OUTPUT

```
terraform output instance_ips
  -> ["34.201.10.21", "34.201.55.10"]
Generated inventory [webservers] section uses those IPs directly.
```

## TERRAFORM (PROVISIONING)

- Cloud resources (VMs, VPCs, subnets, security groups).
- Idempotent infrastructure lifecycle.
- Focus: Infrastructure as Code.

## ANSIBLE (CONFIGURATION)

- Installs packages, configures files, users, services.
- Deploys applications, enforces policy.
- Focus: Configuration as Code.

## BEST PRACTICES FOR USING TOGETHER

- Keep responsibilities separate and clear.
- Generate inventory automatically from Terraform outputs.
- Run Ansible only after infrastructure is successfully created.
- Design for idempotency in both Terraform and Ansible.

## COMMON USE CASES

- **Spin up + configure servers**
- **CI/CD infra + app deploy**
- **Multi-tier app stacks**
- **Ephemeral test environments**

**KEY TAKEAWAY** Terraform builds it. Ansible configures it. Together they automate the full lifecycle -- faster, safer, and more consistently than either tool alone.

# INFRASTRUCTURE AUTOMATION BEST PRACTICES

*Good automation is not just about tools -- it's about the right practices, processes, and people working together.*

### 1 Version Control Everything

Store all code (Terraform, Ansible, scripts, configs) in Git with branching and pull requests.

### 2 Modularize and Reuse

Break configurations into small, reusable modules or roles to reduce duplication.

### 3 Use Variables & Parameters

Avoid hardcoding values -- use variables, tfvars, and inventory/group vars.

### 4 Follow Least Privilege

Grant only the minimum permissions needed for users, service accounts, and tools.

### 5 Validate and Test Early

Use linters, validate, and plan/check mode before applying changes.

### 6 Automate Testing & CI/CD

Run automated tests and pipelines for plan, test, and deployment.

### 7 Keep State Secure

Store Terraform state remotely with encryption and locking -- protect sensitive data.

### 8 Observability & Docs

Enable logging/metrics for automation runs; document decisions and runbooks.

**KEY TAKEAWAY** Golden Rule: automate for consistency, validate for safety, monitor for reliability, improve for the future. Common anti-pattern: manual changes in production.

# TRY IT YOURSELF — EXERCISES 4-6: AI REVIEW DISCIPLINE

*Capstone: constrain an AI prompt, outline the AI review record, and analyze cost/exposure risk -- the final gate before Lab 45 begins.*

## STEPS (notes/lab45-ai-*.md and notes/lab45-cost-exposure-quiz.md)

| Exercise | What To Do |
|---|---|
| **4 — AI Prompt TODOs** | Fill a bounded prompt template (goal, environment, must-include, must-forbid, assumptions) and write one AI suggestion you would reject. |
| **5 — AI Review Record** | Outline docs/ai-iac-review.md fields: prompt, generated excerpt, human corrections, validation evidence, rejected item, residual risks, approval. |
| **6 — Cost/Exposure Quiz** | For public DB, open security group, oversized VM, and missing tags -- write why each fails the contract. |

## AI prompt TODO template to fill

```
Goal: _____          Environment: _____          Must include: _____
Must forbid: secrets, public DB, _____          Assumptions: _____
Output files: infra/terraform/*.tf, infra/ansible/site.yml
```

**KEY TAKEAWAY** TRY IT NOW — Complete Exercises 4-6 before continuing: exercises/exercise-04/05/06-*.md (workspace: examples/module-45-exercises). Final gate before Lab 45.

# LAB OVERVIEW — IaC WITH AI ASSISTANCE

*Use AI to draft Terraform + Ansible sketches for Northstar CRM non-production environments, then validate, correct, and document every decision.*

## BUSINESS SCENARIO

- Leadership freezes an IaC rule: AI may accelerate scaffolding, but humans remain accountable for exposure, cost, state security, and idempotence.
- Syntactically valid Terraform that opens a public database still fails the lab; planned apply without reading the plan fails the lab.
- You draft non-production crm-dev/crm-test sketches so later CI/CD promotions land on predictable namespaces and hosts.

## LEARNING OBJECTIVES

- Write bounded IaC prompts stating assumptions and forbidding secrets/public DBs.
- Review generated Terraform and Ansible critically -- not just "it linted".
- Model variables, validation, providers, and outputs safely.
- Create idempotent Ansible tasks with modules, handlers, and ownership/modes.
- Run format, validate, lint, and plan checks and interpret create/destroy risk.
- Document AI assumptions, corrections, residual risks, and approval status.

## WHAT YOU BUILD

- lab45-crm under ~/java-bootcamp/examples/: infra/terraform/*.tf (versions, providers, variables, main, outputs), terraform.tfvars.example, infra/ansible/site.yml with templates/, inventory.example.yml, and docs/ai-iac-review.md with entry lab45-001 -- prompt, corrections, validation evidence, and at least one rejected or hardened AI suggestion.

**KEY TAKEAWAY** Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) are app-only -- never in .tf/Ansible vars as PII. Estimated time: 2-3 hours, Intermediate difficulty.

# LAB TASKS AND DELIVERABLES

*lab45-crm -- implementation steps, deliverables, and the evaluation rubric.*

| # | Implementation Step |
|---|---|
| 1 | Define the infrastructure contract: env, region, network, runtime, DB posture, tags, cost limits, forbidden exposures. |
| 2 | Draft with constrained AI prompts: small modules, explicit assumptions, no secrets/public DB. Archive prompt as lab45-001. |
| 3 | Review Terraform structure: separate providers/variables/resources/outputs; pin provider versions. |
| 4 | Secure state and inputs: mark sensitive variables, .tfvars.example only, update .gitignore. |
| 5 | Validate the plan: fmt, init -backend=false, validate, plan -- read every create/update/destroy action. |
| 6 | Draft Ansible configuration: modules over shell, handlers, ownership/modes; inventory.example.yml only. |
| 7 | Test idempotence: --syntax-check, ansible-lint, and a second run showing zero changes (or documented substitute). |
| 8 | Write the AI review record: prompt, corrections, validation evidence, one rejected/hardened item, approval. |

## EXPECTED DELIVERABLES

- infra/terraform/*.tf (structured, pinned providers).
- terraform.tfvars.example (safe placeholders only).
- infra/ansible/site.yml + inventory.example.yml.
- docs/ai-iac-review.md with corrections + evidence.
- Plan/lint evidence; no secrets or state files committed.

## RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/AI Review 10 · Docs 10

**KEY TAKEAWAY** Blind apply of AI output is an honor violation. Committed state/secrets must be remediated before scoring. AI used with no review log loses AI/security marks.

# MODULE 45 SUMMARY

*In this module, we explored how Infrastructure as Code helps provision and manage infrastructure consistently, securely, and at scale -- with AI assistance to accelerate productivity.*

## KEY CONCEPTS COVERED

### Infrastructure as Code

Define infrastructure in code for versioning, review, reuse, and automation.

### Terraform

Provision cloud resources declaratively and manage them through state.

### Ansible

Configure and manage systems to the desired state using playbooks.

### End-to-End Automation

Combine Terraform and Ansible to automate provisioning, configuration, and validation.

### AI Assistance

Use AI to draft, explain, troubleshoot, and optimize IaC -- with humans reviewing every decision.

### Reliability & Consistency

Idempotent operations, validation, and best practices ensure consistent outcomes.

## MODULE FLOW RECAP

1. **Design (+ AI)**
2. **Provision (TF)**
3. **Inventory**
4. **Configure (Ansible)**
5. **Validate & Document**

**KEY TAKEAWAY** Infrastructure as Code, combined with configuration management and AI assistance, enables teams to deliver infrastructure that is secure, consistent, and scalable -- with speed and confidence.

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of Infrastructure as Code, Terraform fundamentals, and idempotent automation.*

**1. What is Infrastructure as Code (IaC)?**

**A.** Managing infrastructure using manual steps

**B.** Managing infrastructure through graphical user interfaces

**C. Defining and managing infrastructure using code**

**D.** Storing infrastructure in spreadsheets

**2. Which tool is primarily used for provisioning cloud infrastructure?**

**A.** Ansible

**B.** Docker

**C. Terraform**

**D.** Jenkins

**3. Which tool is best suited for configuration management and application deployment?**

**A.** Terraform

**B. Ansible**

**C.** Git

**D.** AWS CloudFormation

**4. What does "idempotent" mean in automation?**

**A.** Running a task once only

**B.** Getting a different result each time

**C. Multiple runs produce the same desired state**

**D.** Manual approval is required every time

**KEY TAKEAWAY** IaC defines and manages infrastructure using code; Terraform provisions; Ansible configures; idempotent means multiple runs produce the same desired state.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions on Terraform configuration, commands, AWS security groups, and Ansible modules.*

**5. Which file defines Terraform resources?**

**A.** inventory.ini
**B.** playbook.yml
**C. main.tf**
**D.** hosts.yml

**6. Which command initializes a Terraform working directory?**

**A.** terraform plan
**B. terraform init**
**C.** terraform apply
**D.** terraform destroy

**7. What is the purpose of a security group in AWS?**

**A.** Manage IAM users
**B. Control inbound and outbound traffic**
**C.** Store encrypted backups
**D.** Manage billing alerts

**8. Which Ansible module would you use to install a package on a Linux server?**

**A.** copy
**B.** service
**C. yum**
**D.** template

**KEY TAKEAWAY** main.tf defines Terraform resources; terraform init initializes the working directory; a security group controls inbound/outbound traffic; yum installs packages on Linux.

*"Automate. Provision. Configure. Build infrastructure faster, smarter, at scale."*

MODULE 45 COMPLETE

# Infrastructure as Code with AI Assistance

You can now provision infrastructure with Terraform, configure it with Ansible, and use AI assistance responsibly -- with humans accountable for every decision.